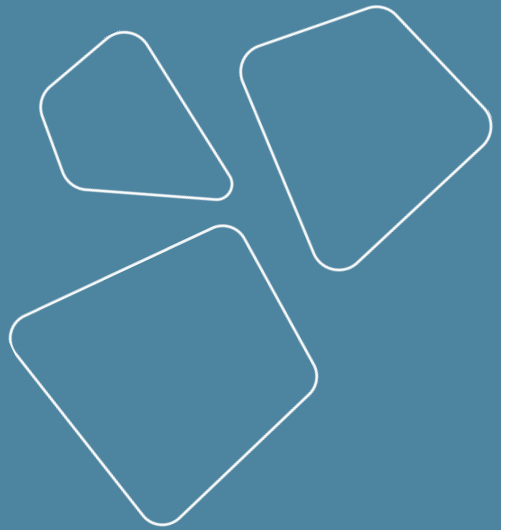
Python tools

Vladislav Goncharenko

spring 2025

Revise



- Python project structure
- Managing environments
- Managing dependencies
- CLI
- Changelog

Python project structure

girafe
ai

01

Modules in Python

A module is a file containing Python definitions and statements.

Definitions from a module can be imported into other modules or into the main module.

```python
# Fibonacci numbers module

def fib(n):    # write Fibonacci series up to n
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):   # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

```
>>> import fibo

>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

Packages

- python package structure
  - common extensible format
  - able to be packaged
- subject to build systems

More on python packaging

```
sound/                          Top-level package
      __init__.py               Initialize the sound package
      formats/                  Subpackage for file format conversions
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
              auwrite.py
              ...
      effects/                  Subpackage for sound effects
              __init__.py
              echo.py
              surround.py
              reverse.py
              ...
      filters/                  Subpackage for filters
              __init__.py
              equalizer.py
              vocoder.py
              karaoke.py
              ...
```

pyproject.toml

Standard file format for modern package metadata instead of many older files.

Packaging user guide, Pip documentation on pyproject

Set of tools supporting pyproject.toml

```toml
[project]
name = "mypackage"
version = "2023.05.1"
description = "A package for demonstrating Python packaging"

[build-system]
requires = ["setuptools >= 61.0.0"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]
```

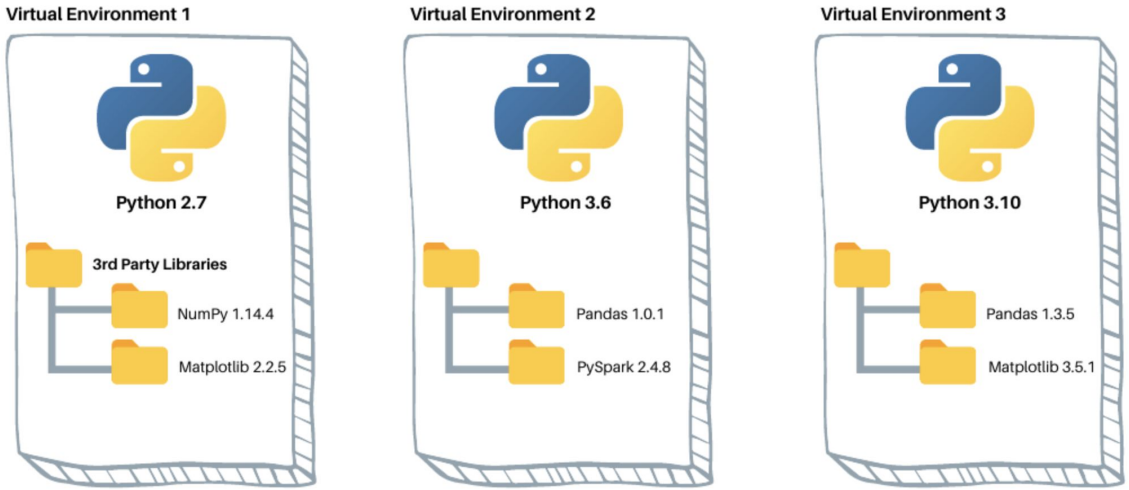
Environments

girafe
ai

02

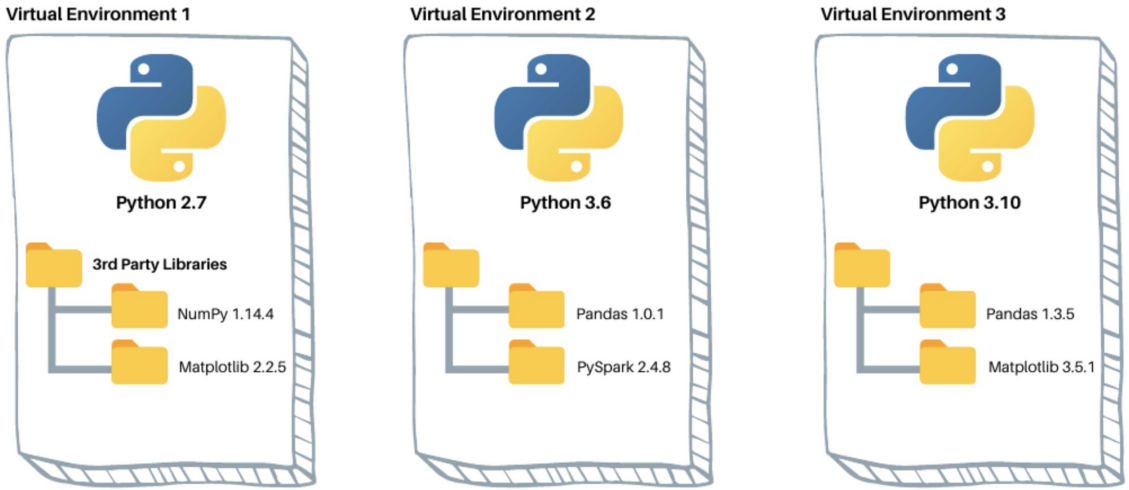
Runtime

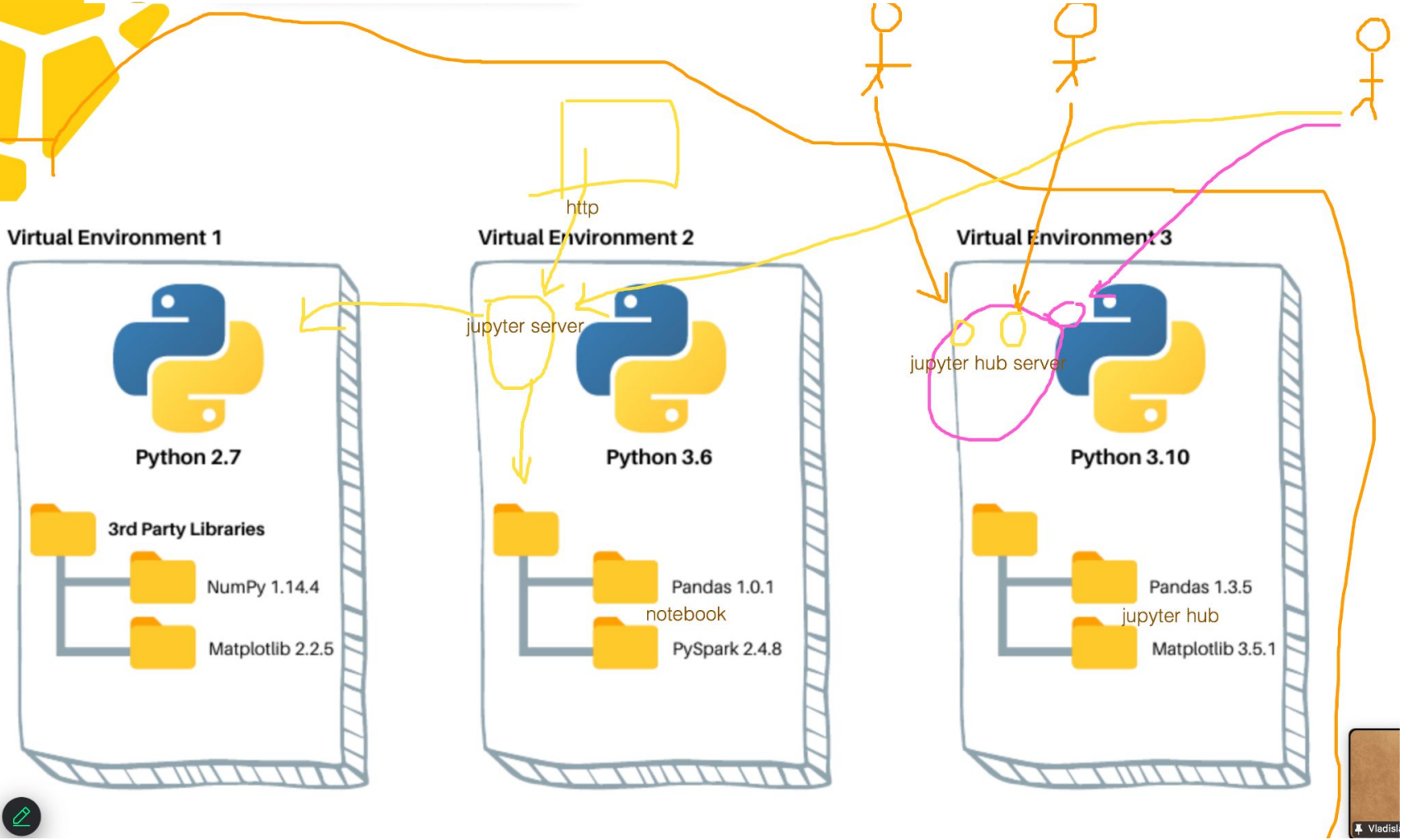
- To execute your code you need some runtime
- Python is a dynamic language, so we have many versions
- But we have many projects
- And we need to run it on the same machine



What do we want from environments?

- Create environments
  - separate for each project on one host machine
- Remember and install packages
  - repeatable set of packages for each project




http
Virtual Environment 1
Python 2.7
3rd Party Libraries
NumPy 1.14.4
Matplotlib 2.2.5
Virtual Environment 2
jupyter server
Python 3.6
Pandas 1.0.1
notebook
PySpark 2.4.8
Virtual Environment 3
jupyter hub server
Python 3.10
Pandas 1.3.5
jupyter hub
Matplotlib 3.5.1

Vanilla solution for environments

- Create environments
  - `python -m venv /path/to/new/virtual/environment`
- Remember and install packages
  - `pip freeze > requirements.txt`
  - `pip install -r requirements.txt`

Any problems with this?

- python version not saved
- only python dependencies are managed
- long and structureless requirements file (no dependencies)
- no groups of dependencies (prod vs dev)
- not portable to all OSes

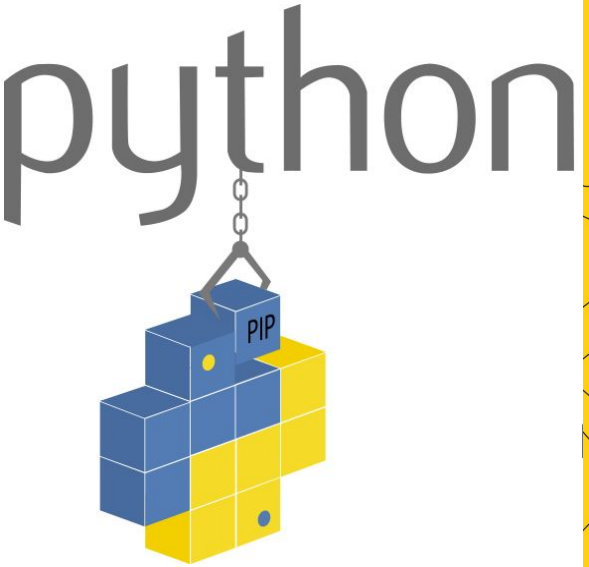
Managing environments

- venv - Python standard library
- virtualenv - extended toolkit for virtual environments
- poetry
- conda - able to install non python dependencies
  - miniconda
  - mamba
- pyenv - automatic envs switching (good article)
- pipx
- pipenv
- uv - super fast tool written in Rust

More on virtual envs

Managing dependencies

- pip (via requirements.txt)
- poetry - convenient tool covering most needs
- pip-tools
- Pipfile (via pipenv)
- conda
  - `conda env export > environment.yml`
- uv - super fast tool written in Rust



Build systems

- setuptools
- poetry
- Hatchling
- flit
- ...

Poetry capabilities

- dares to install ONE tool per OS
  - also available for CI installation
- manages environments (optional)
- saves Python version
- resolves conflicts better due to hierarchical dependency
- supports groups of dependencies
- manages dependencies for all platforms simultaneously
- additional package indexes made easy

https://python-poetry.org/docs/

Nice tutorial on Poetry packaging

Poetry usage

Creates two files:

- `pyproject.toml` - initial dependencies, config for all tools
- `poetry.lock` - all concrete dependencies (platform agnostic)

If using conda environments:

```
poetry config virtualenvs.create false
```

For installing dependencies:

```
poetry install
```

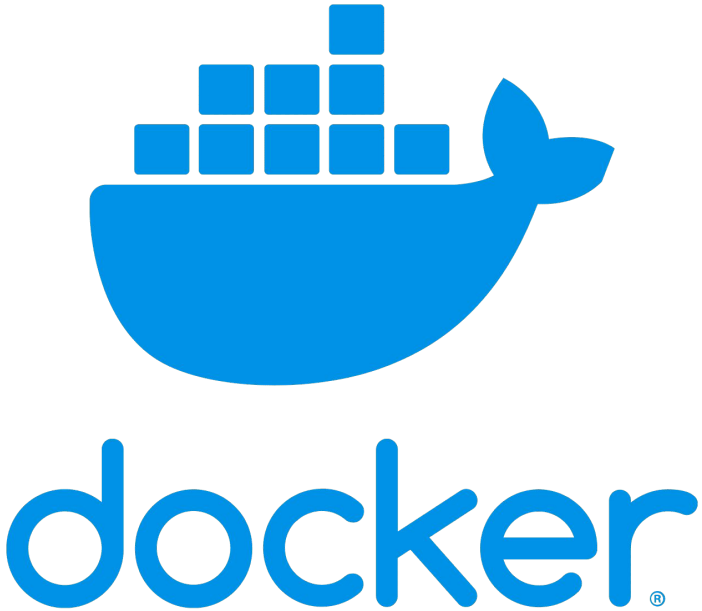
Also it is able to build and publish to PyPI or your own package registry

```
poetry build && poetry publish
```

Docker

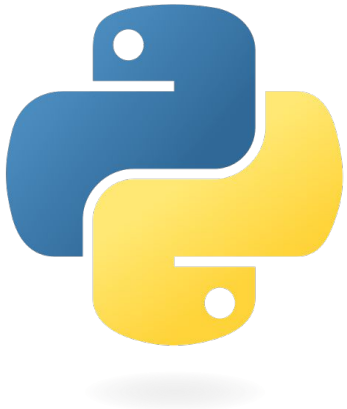
As environment for Python:

- No change of OS
- No change of Python
- Dependencies compiled into image
- Pure pip can be used



Package repositories

- PyPI
  - can mirror public version
- custom
  - Artifactory
  - etc.



PyPi

Command Line Interface

girafe
ai

03

Command Line Interfaces (CLI)

Default tool for CLI in Python is argparse

Which is clone of C library and is painful to use

```
import argparse

parser = argparse.ArgumentParser(description='Process some integers.')
parser.add_argument('integers', metavar='N', type=int, nargs='+',
                    help='an integer for the accumulator')
parser.add_argument('--sum', dest='accumulate', action='store_const',
                    const=sum, default=max,
                    help='sum the integers (default: find the max)')

args = parser.parse_args()
print(args.accumulate(args.integers))
```

CLIs for humans

Also there are several tools made for humans.

One of them is Fire.
It makes CLI out of pure
Python functions and classes.

```python
import fire

def hello(name="World"):
  return "Hello %s!" % name

if __name__ == '__main__':
  fire.Fire(hello)
```

Then, from the command line, you can run:

```
python hello.py  # Hello World!
python hello.py --name=David  # Hello David!
python hello.py --help  # Shows usage information.
```

Other options:
https://click.palletsprojects.com/

Changelog

Changelog generation:
https://towncrier.readthedocs.io/en/stable/

Package versioning

Main method nowadays is SemVer standard

https://semver.org/

Reference projects

1. Template project with basic setup
   https://github.com/v-goncharenko/data-science-template
2. Good example of project with most of tools applied
   https://github.com/v-goncharenko/mimics/
3. Dummy project for this course
   https://github.com/vlad21naumov/cats-and-dogs
4. Full fledged library made by biologists
   https://github.com/NeuroTechX/moabb

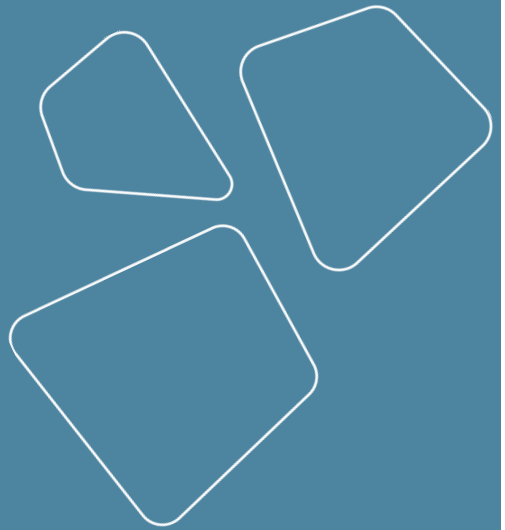
How to write a code

We will talk about it next time, now some great references:

- Summary of the Clean Code book
  https://gist.github.com/wojteklu/73c6914cc446146b8b533c0988cf8d29
- Same thing adopted for Python
  https://github.com/zedr/clean-code-python
- And for ML
  https://github.com/davified/clean-code-ml

There's no silver bullet in code writing,
so despite all the books you need to think yourself.

Revise



Thanks for attention!

Questions?

girafe ai